
UFCS is a breaking change, of the absolutely worst kind

A part zero general remark

I'm sure some of you will read this paper, and have hasty reactions thinking "no, no, it doesn't do that, you have misunderstood the proposal, it will look for a member **first** *and only then* look for a free function, so the problems you're concerned about don't arise".

They do. That doesn't help **at all**. So please read this paper carefully, and keep your mind, eyes, and ears open.

Contemplate what it says, and avoid those hasty reactions.

Abstract

This paper explains why C++ should never, not ever, adopt a Unified Function Call Syntax that can turn (syntactic) member function calls into calls of free functions, possibly using ADL.

The proposal [P3021](#) (correctly) says thus:

Q: Is this fully backward-compatible without breaking existing code?

A: Yes

But what that means is not a feature, it's a bug, it's a HUGE bug.

Correct, it doesn't "break" your existing code in a way that would turn it from well-formed to ill-formed. And that's a problem, because **when that code changes meaning**, it may continue to compile, with a different meaning.

And that in and of itself is not even the biggest problem. That proposal breaks an existing guarantee that all existing C++ has, or thought it has, a guarantee that a sizable portion of our users rely on, some as a happy accident (but such a happy accident makes them very happy), and some as a deathly serious and intentional design choice:

It breaks the guarantee that code that uses member function calls will *never* be subject to any of the complexity and woes of ADL.

We do have proposals trying to put that guarantee into good use: see [P2855](#) for an attempt to tame the effects of ADL on how to write Customization Points (and thing that aren't customizations, but just opt-ins) for P2300. The

UFCS proposal makes approaches like that no longer work.

Or, as another phrasing: it doesn't break the well-formedness of code. Even when it should. It doesn't break short-term "programming", as defined as a notion by a certain Dr. Winters.

But it breaks "**engineering**", as defined by the same person.

Because it breaks the meaning of code across refactorings, possibly silently accepting a program that wasn't intended to be accepted, giving it a meaning that is not at all what a programmer intended.

Some remarks on the history of UFCS

As far as I understood, multiple members of WG21 opposed UFCS for this sort of reasons. I doubt they expressed the rationale sufficiently crisply. I vividly recall "it breaks future code" being stated and to some extent explained.. ..but I do not recall it being clearly pointed out that we're breaking a major and massively significant design guarantee, or that the future breakages are far worse than they might immediately appear, because they may be *silent*.

Some breakage examples

A rename of a member function

I trust most of us have done this sort of a refactoring a thousand times. You simply have

```
struct Foo {  
    void snap();  
};
```

and then calls to that in any which context,

```
template <class T>  
void do_snap(T&& f) {  
    f.snap();  
}
```

you then decide to rename snap() to slap().

Fine, you rename it in the library that declares and defines Foo. And then you rebuild the users, and if any one of them is wrong, the compiler will tell you. This is literally the bread-and-butter of a statically type-checked language, you can do this at arbitrary scales.

Right, now some of you might (or more likely will) say "just use an IDE". The problem is that that's not a sufficient answer. You won't always have all the declarations/definitions and uses in the same project for refactoring browsers to find them all. I can, for example, safely assure you that I won't have all of Qt loaded in my IDE when I want to refactor something in Qt Core, nor will any of you have all the applications loaded in your IDE when you refactor one of your libraries.

Right, now, the problem; with the proposed UFCS, you might not get an error that snap() was renamed to slap(). It might continue to compile, finding a free function, arbitrarily far away in some code you never wrote, brought in by any of the many associated namespaces that might have that effect. In other words, The Woes of ADL.

And then it's anyone's guess whether you ever intended a function like that to be called by your code. For a lot of code, it's today safe to assume that you certainly didn't, because you used a call syntax that's guaranteed not to go

looking for an ADL overload set.

A removal of a member function, or never writing one

I think this example has been brought up before, but I want to reiterate it regardless. Somewhat like before, with some small changes, we have

```
struct Foo {  
    void slap();  
};
```

and then calls to that in any which context,

```
template <class T>  
void slap(T&& f) {  
    f.slap();  
}
```

Of course, those with keen eyes will notice that we have a free function template `slap()` which calls a member `slap()`. Perfectly reasonable, perfectly fine, not worrisome by any means.

Let's add a class:

```
struct Bar {  
};
```

Okay. You have existing calls of the free function template `slap()` with an argument of type `Foo`. Now add some with an argument of type `Bar`, or just add calls of some template that ends up calling the free function template `slap()` with a `Bar` argument, when it already called `slap()` with a `Foo` argument.

The added call, via whichever layers and forwarders, is infinitely recursive. The same thing happens if you have an existing member `slap()` and you for whatever reason remove it.

If that doesn't scare the living daylights out of you, I wonder what will.

We have here some refactoring that seem perfectly reasonable, and the first part is actually no refactoring at all, it's just a new type, and you're going to introduce it into the system. Write the class, start using it. See if the code using it compiles.

And then it silently compiles and explodes your program. Sure, fine, unit tests, release tests, smoke tests, all of that - but the problem is hideously difficult to grasp for a non-expert, and it's a terrible prospect to have a problem like that sneak past any of those testing rounds.

Changing the type of a function parameter

Next, we have a class that has an innocent `free()` function, and we call it in a function that takes a reference to our class type:

```
struct Kraken {  
    void free();  
};  
  
void release(Kraken& k) {
```

```
        k.free();  
    }
```

We then change the processing function to take the parameter by pointer:

```
void release(Kraken* k) {  
    if (k)  
        k.free();  
}
```

We forgot to change the call from `k.free()` to `k->free()`. The language won't point out this little problem, and will be looking for a non-member `free()` function to call instead.

A variant of this theme is changing

```
void cleanup(X& x) {  
    x.close();  
}
```

to

```
void cleanup(int x) {  
    x.close();  
}
```

These last two examples end up deallocating memory and closing a file handle, and it's unlikely that was the intent of the code.

Making a member inaccessible, or deleting a member

We end with the same result as removing a member if we tighten a member's access or `=delete` it.

We have relatively consistently avoided changing the meaning of code on such access changes.

And for a deleted member, that's so opposite to how deleted members should work that it's no longer funny - we have a perfect match, lookup finds it, overload resolution resolves to it - the call should be ill-formed, not dispatch to something else. This is just fundamentally unlike how a deleted overload is supposed to behave.

None of these problems are specific to generic code that uses templates

Sure, I used templated callers in those examples. But the problems may arise without any templates. The use of templates there is a subtle hint; template parameters bring in additional associated namespaces. For those familiar with the woes of ADL, this isn't new, that's just more namespaces that ADL may look into to successfully find a non-member function to call with your member call syntax, once UFCS enters the building.

A non-recap on the woes of ADL

I'm actually not going deep into the woes of ADL. Whether accidentally or deliberately, many users completely and categorically avoid its complexity by using straight-up OOP code with member functions and their call syntax thereof. It's wildly and widely successful; it's easily comprehensible to millions of programmers to explain "here's the type of your thing, here's its data, here are the functions that manipulate it, all in one cohesive package". And as a bonus, none of the long-distance written-by-someone-else functions mess with any of that.

The proposal for making member-syntax calls reach into non-member functions breaks that simplicity and comprehensibility. That's not a service to our users, that's a devastating and horrible disservice to our users. Please don't inflict this horror upon them.

Some responses to the Motivation parts of the UFCS proposal

"Enable generic code"

Sure, UFCS would allow writing multi-meaning generic code that works the same way with both member functions and non-member functions. But it also dilutes the ability to reason about such code. Considering what ADL can do, and what functions you end up calling, that genericity isn't necessarily what you want, so it's much harder to reason about the meaning of code.

Yeah, we could introduce new syntaxes for "member-only" or "free function only" calls, but we already have those meanings, and they are in widespread use. The ability to call members and only members got there first. Don't break it.

"Enable encapsulated code: Nonmember nonfriends increase encapsulation, lower coupling"

The reasons stated in this one are all technically lofty and good.. but they are also hard to understand for users accustomed to how member functions work everywhere, and how to design and program with simple classes. They don't just get a benefit out of this, they get increased complexity where none was appropriate. So despite all the technical wins here, I disagree on this one being a pure win, it has downsides as well.

It's also not like tightly-coupled member functions having unrestricted access to all of your data members are always an encapsulation problem. In some over-simplified examples it might seem that way, but in practice, many of those data members are themselves of class type, so there is encapsulation in the code even if you don't ever use a single non-member function.

"It's friendlier to programmers: Discoverability"

Well, first of all, I disagree with this sort of UFCS being friendly to anyone, expert or non-expert, considering that it removes a fundamental guarantee and replaces it with something that you can't reason about at all, let alone reason about it simply.

The additional specific part with which I disagree with is the code completion discoverability, and "to programmers to discover what they can do next in their code". It's already sometimes a problem to show umpteen completion choices for "what they can do next", adding more is not a pure improvement. And in various situations, showing more completions that allow you to invoke functions you're actually never supposed to use for your particular problem isn't an improvement.

Summa summarum

Enabling a multi-meaning for a member function call to also find and call non-member calls without changing the syntax of those calls is a terrible idea at such a level that words fail me. WG21 should not do this disservice to its users. If we want to introduce such a multi-meaning facility, we **have to** find a new syntax for it, despite how icky that might seem to some. Otherwise we're hoping for a magical improvement, but that magic ends up being the thing that brings forth non-improvements that make those improvements void.

The way I see this whole discussion is that we want to enable terse and simple and uniform code. While that's a lofty goal, and could theoretically serve some non-experts well in addition to experts, it ends up not doing so

when it introduces hideous complexity in places that were previously guaranteed to not have such complexity.

We would certainly serve experts. We would probably serve some non-experts. But at the same time, we expose many programmers to a possibly huge amount of added complexity just dropped into their code, whether existing or new, with completely unpredictable and hard-to-reason results. Except some such results can be exemplified to be seriously undesirable, and have the most horrible change of meaning, like turning compile-time errors into run-time errors, including logic errors, infinite loops and crashes.

The overall benefit/cost ratio isn't there. That ratio is terrible. Don't do it.